AG1 - Actividad Guiada 1

Nombre: Miguel Laversa

En esta actividad aplico algoritmos voraces, divide y vencerás, vuelta atrás y programación dinámica. Además de ejecutar los ejemplos guiados, comparo soluciones y analizo su complejidad.

1. Torres de Hanói - Divide y vencerás

El problema se divide en dos subproblemas de tamaño `n-1`. La recurrencia es $T(n) = 2T(n-1) + 1$, por lo que se realizan $2^n - 1$ movimientos y la complejidad temporal es $O(2^n)$.

In [1]:
```python
def torres_hanoi(n, origen, destino, auxiliar):
    if n == 0:
        return []
    # El disco mayor solo puede moverse después de liberar los n-1 superiores.
    return (
        torres_hanoi(n - 1, origen, auxiliar, destino)
        + [(origen, destino)]
        + torres_hanoi(n - 1, auxiliar, destino, origen)
    )

movimientos = torres_hanoi(3, 1, 3, 2)
for numero, (origen, destino) in enumerate(movimientos, start=1):
    print(f"{numero}: mover un disco de {origen} a {destino}")
print("Total de movimientos:", len(movimientos))
```

```
1: mover un disco de 1 a 3
2: mover un disco de 1 a 2
3: mover un disco de 3 a 2
4: mover un disco de 1 a 3
5: mover un disco de 2 a 1
6: mover un disco de 2 a 3
7: mover un disco de 1 a 3
Total de movimientos: 7
```

2. Sucesión de Fibonacci

La versión recursiva muestra con claridad la descomposición, pero repite subproblemas y tiene complejidad exponencial. La versión con memoria conserva los resultados intermedios y reduce el coste temporal a $O(n)$.

In [2]:
```python
from functools import lru_cache

def fibonacci_recursivo(n):
    if n < 2:
        return n
    return fibonacci_recursivo(n - 1) + fibonacci_recursivo(n - 2)

@lru_cache(maxsize=None)
def fibonacci_memoria(n):
    # La caché evita resolver varias veces los mismos subproblemas.
    if n < 2:
        return n
    return fibonacci_memoria(n - 1) + fibonacci_memoria(n - 2)

print("Fibonacci(10), recursivo:", fibonacci_recursivo(10))
print("Fibonacci(40), con memoria:", fibonacci_memoria(40))
```

```
Fibonacci(10), recursivo: 55
Fibonacci(40), con memoria: 102334155
```

3. Devolución de cambio - Algoritmo voraz

En cada etapa se toma el mayor número posible de monedas del valor actual. El procedimiento es $O(k)$ para `k` tipos de moneda, pero no garantiza el óptimo para cualquier sistema monetario.

In [3]:
```python
def cambio_voraz(cantidad, monedas):
    if cantidad < 0 or any(moneda <= 0 for moneda in monedas):
        raise ValueError("La cantidad y las monedas deben ser válidas")
    solucion = []
    restante = cantidad
    # Ordenar de mayor a menor implementa la decisión voraz.
    for moneda in sorted(monedas, reverse=True):
        unidades, restante = divmod(restante, moneda)
        solucion.append((moneda, unidades))
    return solucion, restante

for cantidad, monedas in [(27, [25, 10, 5, 1]), (15, [11, 5, 1])]:
    solucion, restante = cambio_voraz(cantidad, monedas)
    total_monedas = sum(unidades for _, unidades in solucion)
    print(cantidad, solucion, "monedas usadas:", total_monedas, "restante:",
restante)
```

```
27 [(25, 1), (10, 0), (5, 0), (1, 2)] monedas usadas: 3 restante: 0
15 [(11, 1), (5, 0), (1, 4)] monedas usadas: 5 restante: 0
```

Para 15 unidades y monedas `[11, 5, 1]`, el algoritmo devuelve una moneda de 11 y cuatro de 1. Son cinco monedas, mientras que el óptimo son tres monedas de 5. La decisión local no conduce siempre al óptimo global.

4. N-Reinas - Vuelta atrás

La solución se construye columna a columna. Una rama se descarta en cuanto una reina coincide en fila o diagonal con alguna de las anteriores.

In [4]:
```python
def es_prometedora(solucion, fila):
    # El índice representa la columna y el valor representa la fila, ambos
desde cero.
    columna = len(solucion)
    return all(
        fila != fila_anterior
        and abs(fila - fila_anterior) != columna - columna_anterior
        for columna_anterior, fila_anterior in enumerate(solucion)
    )

def resolver_reinas(n):
    soluciones = []

    def explorar(solucion):
        if len(solucion) == n:
            soluciones.append(tuple(solucion))
            return
        for fila in range(n):
            # Solo se amplían las soluciones parciales que aún pueden ser
válidas.
            if es_prometedora(solucion, fila):
                explorar(solucion + [fila])
```

```
    explorar([])
    return soluciones

soluciones_4 = resolver_reinas(4)
soluciones_8 = resolver_reinas(8)
print("Soluciones para 4 reinas:", soluciones_4)
print("Número de soluciones para 8 reinas:", len(soluciones_8))
```

```
Soluciones para 4 reinas: [(1, 3, 0, 2), (2, 0, 3, 1)]
Número de soluciones para 8 reinas: 92
```

5. Viaje por el río - Programación dinámica

El coste mínimo entre dos embarcaderos se construye reutilizando costes ya calculados. La tabla evita repetir subproblemas y permite reconstruir la ruta elegida.

In [5]:

```
import math

TARIFAS = [
    [0, 5, 4, 3, math.inf, math.inf, math.inf],
    [math.inf, 0, math.inf, 2, 3, math.inf, 11],
    [math.inf, math.inf, 0, 1, math.inf, 4, 10],
    [math.inf, math.inf, math.inf, 0, 5, 6, 9],
    [math.inf, math.inf, math.inf, math.inf, 0, math.inf, 4],
    [math.inf, math.inf, math.inf, math.inf, math.inf, 0, 3],
    [math.inf, math.inf, math.inf, math.inf, math.inf, math.inf, 0],
]

def viaje_rio(tarifas):
    n = len(tarifas)
    costes = [fila[:] for fila in tarifas]
    siguiente = [[None] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if math.isfinite(tarifas[i][j]):
                siguiente[i][j] = j
    # Se resuelven primero los trayectos cortos para reutilizarlos en los
largos.
    for longitud in range(2, n + 1):
        for origen in range(n - longitud + 1):
            destino = origen + longitud - 1
            for intermedio in range(origen + 1, destino):
                candidato = costes[origen][intermedio] + costes[intermedio]
[destino]
                if candidato < costes[origen][destino]:
                    costes[origen][destino] = candidato
                    siguiente[origen][destino] = siguiente[origen][intermedio]
    return costes, siguiente

def reconstruir_ruta(siguiente, origen, destino):
    if siguiente[origen][destino] is None:
        return []
    ruta = [origen]
    while origen != destino:
        origen = siguiente[origen][destino]
        ruta.append(origen)
    return ruta

costes, siguiente = viaje_rio(TARIFAS)
print("Coste mínimo de 0 a 6:", costes[0][6])
print("Ruta óptima:", reconstruir_ruta(siguiente, 0, 6))
```

```
Coste mínimo de 0 a 6: 11
Ruta óptima: [0, 2, 5, 6]
```

La ruta obtenida es `0 → 2 → 5 → 6`, con coste total 11. La tabla requiere $O(n^2)$ de espacio y el cálculo tiene coste $O(n^3)$.

6. Ejercicio adicional - Par de puntos más cercanos

Comparo fuerza bruta con divide y vencerás. En una dimensión basta ordenar y revisar elementos consecutivos. En dos dimensiones se divide por la coordenada `x` y se revisa una franja alrededor de la separación. Para tres dimensiones utilizo un árbol `KD`, una estructura que también divide recursivamente el espacio.

In [6]:

```python
import math
import random
import time
from scipy.spatial import cKDTree

def par_cercano_fuerza_bruta(puntos):
    mejor_distancia = math.inf
    mejor_par = None
    for i in range(len(puntos)):
        for j in range(i + 1, len(puntos)):
            distancia = math.dist(puntos[i], puntos[j])
            if distancia < mejor_distancia:
                mejor_distancia = distancia
                mejor_par = (puntos[i], puntos[j])
    return mejor_distancia, mejor_par

def par_cercano_1d(puntos):
    ordenados = sorted(puntos)
    distancia, par = min(
        (ordenados[i + 1] - ordenados[i], (ordenados[i], ordenados[i + 1]))
        for i in range(len(ordenados) - 1)
    )
    return distancia, par

def par_cercano_2d(puntos):
    def resolver(ordenados_x):
        n = len(ordenados_x)
        if n <= 3:
            return par_cercano_fuerza_bruta(ordenados_x)
        mitad = n // 2
        x_medio = ordenados_x[mitad][0]
        izquierda = resolver(ordenados_x[:mitad])
        derecha = resolver(ordenados_x[mitad:])
        mejor_distancia, mejor_par = min(izquierda, derecha, key=lambda
resultado: resultado[0])
        # La franja contiene los pares que podrían cruzar la división.
        franja = sorted(
            (p for p in ordenados_x if abs(p[0] - x_medio) < mejor_distancia),
            key=lambda p: p[1],
        )
        for i, punto in enumerate(franja):
            # En el plano basta revisar los siete puntos siguientes ordenados
por y.
            for candidato in franja[i + 1:i + 8]:
                if candidato[1] - punto[1] >= mejor_distancia:
                    break
                distancia = math.dist(punto, candidato)
                if distancia < mejor_distancia:
```

```
                    mejor_distancia, mejor_par = distancia, (punto, candidato)
        return mejor_distancia, mejor_par

    return resolver(sorted(puntos))

def par_cercano_kdtree(puntos):
    arbol = cKDTree(puntos)
    # El primer vecino es el propio punto; el segundo es el candidato buscado.
    distancias, indices = arbol.query(puntos, k=2)
    i = min(range(len(puntos)), key=lambda indice: distancias[indice][1])
    j = int(indices[i][1])
    return float(distancias[i][1]), (puntos[i], puntos[j])
```

In [7]:

```
generador = random.Random(42)
puntos_1d = generador.sample(range(1, 100_000), 1_000)
puntos_2d = list({(generador.randrange(100_000), generador.randrange(100_000))
for _ in range(1_200)})[:1_000]
puntos_3d = list({tuple(generador.randrange(100_000) for _ in range(3)) for _
in range(1_200)})[:1_000]

inicio = time.perf_counter()
resultado_1d = par_cercano_1d(puntos_1d)
tiempo_1d = time.perf_counter() - inicio

inicio = time.perf_counter()
resultado_2d = par_cercano_2d(puntos_2d)
tiempo_2d = time.perf_counter() - inicio

inicio = time.perf_counter()
resultado_3d = par_cercano_kdtree(puntos_3d)
tiempo_3d = time.perf_counter() - inicio

print(f"1D: distancia={resultado_1d[0]:.4f}, par={resultado_1d[1]}, tiempo=
{tiempo_1d:.6f}s")
print(f"2D: distancia={resultado_2d[0]:.4f}, par={resultado_2d[1]}, tiempo=
{tiempo_2d:.6f}s")
print(f"3D: distancia={resultado_3d[0]:.4f}, par={resultado_3d[1]}, tiempo=
{tiempo_3d:.6f}s")

muestra = puntos_2d[:150]
distancia_bruta, _ = par_cercano_fuerza_bruta(muestra)
distancia_dyv, _ = par_cercano_2d(muestra)
assert math.isclose(distancia_bruta, distancia_dyv)
print("Comprobación 2D frente a fuerza bruta: correcta")
```

```
1D: distancia=1.0000, par=(14663, 14664), tiempo=0.000228s
2D: distancia=71.4703, par=((73116, 47920), (73048, 47942)), tiempo=0.002230s
3D: distancia=390.1871, par=((86006, 68289, 37548), (86099, 67915, 37487)),
tiempo=0.002355s
Comprobación 2D frente a fuerza bruta: correcta
```

Complejidad

Método	Tiempo	Espacio
Fuerza bruta	$O(n^2)$	$O(1)$
Ordenación en 1D	$O(n \log n)$	$O(n)$
Divide y vencerás en 2D	$O(n \log^2 n)$ en esta implementación	$O(n)$
Árbol KD en 3D	$O(n \log n)$ en promedio	$O(n)$

La comprobación con fuerza bruta sobre una muestra confirma que la implementación bidimensional obtiene la misma distancia mínima.